// PEB NEWS

SEMINAR PROJECT · STATUS & TECHNICAL SUMMARY

PEB News

Responsive newsroom frontend and production API integration

A consolidated overview of the current frontend implementation, live API contract, responsive design system, deployment configuration, verification results, known limitations and recommended next steps.

PREPARED
10 November 2026

OVERALL STATUS
Homepage integration complete; production hardening pending

PROJECT FOLDER
C:\Users\hp\Desktop\SEMINAR PROJECT\pebnews

PRODUCTION API
https://pebnews.pebnews.com/api/v1

FRONTEND STACK
React 19, TypeScript, Vite 6, Tailwind CSS 4

SOURCE CONTROL
No Git repository detected

EXECUTIVE SUMMARY

What the project is today

PebNews is a responsive, single-page news homepage built with React, TypeScript, Vite and Tailwind CSS. It now reads live public content from the deployed PebNews API and renders the homepage modules, article-led cards, trending topics, featured categories and API-backed search. The visual system supports a dark default theme with a persistent light mode and was verified across desktop, tablet and mobile widths.

Live	210	15 / 20	390–1440
PRODUCTION API CONNECTED	DEMO ARTICLES AT LAST REVIEW	CATEGORIES / TAGS RETURNED	RESPONSIVE VERIFICATION RANGE

Current delivery position: the homepage is functionally integrated, the production build passes, and a deployable dist/ folder exists. The project is ready for a homepage demo or review deployment, but it is not yet a complete news product because article-detail pages, reader authentication, automated tests and real media handling are still missing.

Status board

AREA	STATUS	CURRENT POSITION
Responsive homepage UI	Complete	All primary homepage modules are implemented and styled.
Production API integration	Complete	Homepage, article list, categories, tags and search are wired.
Typography and visual polish	Complete	Type scale enlarged; dark/light themes and responsive layout verified.
Loading, error and retry states	Complete	Homepage skeleton, error screen and retry control are implemented.
Image resilience	Workaround	Broken remote images fall back to local category artwork.
Deployment configuration	Complete	Vite and Netlify proxies are configured for the public API.
Article detail pages	Not built	Article links remain placeholders; no router is installed.
Reader accounts and bookmarks	Not built	API supports reader auth; the frontend has no login or profile UI.
Video and photo modules	Hidden	The public API exposes no video or gallery endpoints.
Automated tests	None	Verification is manual plus TypeScript/Vite build validation.

Most important issues to resolve next

1. **Backend media URLs are broken:** seeded article image URLs under /storage/articles/ return HTTP 404; the frontend currently masks this with local fallbacks.
2. **Article navigation is absent:** headlines and search results do not yet open article pages.
3. **Reader authentication is unbuilt:** the API supports Sanctum cookie sessions, but the interface still shows a hardcoded “Amaka” account.

PROJECT SCOPE

Scope, structure and user experience

Product scope

The current deliverable is the public homepage of a digital newsroom for PEB News, positioned around independent, credible African journalism with a focus on the Niger Delta. The homepage prioritises editorial hierarchy: breaking news, a lead story, latest news, trending and popular stories, topics, a personalised-style recommendation shelf, featured categories, advertising placements and a newsletter prompt.

Implemented homepage flow

Header and utility bar	Logo, primary navigation, search field and dialog, notifications, account chip, date/location/weather and theme switch.	Static nav + live date
Breaking news ticker	Rotating API-driven breaking headlines with timestamps, previous/next controls and reduced clutter on mobile.	/homepage
Lead story and top rail	Image-led lead article with category, headline, summary, byline and date; four supporting top stories on desktop.	/homepage + /articles
News grid	Latest News, Trending, Popular, Trending Topics, For You, inline newsletter and advertising units.	API + curated slices
Featured categories	Eight visual category tiles with local category artwork and a Nigerian flag accent for The Nation.	/categories
Footer	Brand summary, quick links, company links, newsletter form, app-store prompts, social icons and copyright.	Static

Frontend routes and pages

There is one rendered page: the homepage. The application does not use React Router or any equivalent client-side routing. Category links, article cards, “View all” actions and search results currently point to #. A full news site will require routes for articles, categories, tags, reporters, search results, account screens and bookmarks.

WHAT IS COMPLETE

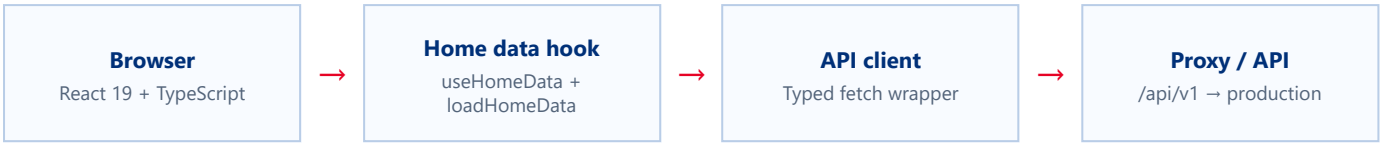
- Responsive homepage composition
- Dark default theme and light mode
- Live public API content
- API-backed search dialog
- Loading, error and retry states
- Broken-image fallback handling
- Deployment proxy configuration

WHAT IS STILL OUT OF SCOPE

- Article-detail rendering
- Category, tag and reporter pages
- Reader registration and login
- Bookmarks and profile management
- Newsletter subscription backend
- Real video and photo galleries
- Editorial trending/popularity signals

TECHNICAL ARCHITECTURE

How the frontend and API fit together



The frontend is intentionally small and dependency-light. App-level data loading is centralised in a custom hook. View components receive typed view models as props and do not fetch independently. This keeps rendering predictable and makes future route-level refactoring straightforward.

Stack and tooling

LAYER	TECHNOLOGY	PURPOSE
UI framework	React 19	Component-based homepage rendering.
Language	TypeScript 5.7	Strict typing for API payloads and view models.
Build tool	Vite 6.4.4	Development server, proxy and production bundling.
Styling	Tailwind CSS 4	Utility-first layout, typography and theme tokens.
Icons	lucide-react	Consistent interface and social icons.
Package manager	pnpm 11	Dependency installation and scripts.
Routing	None	Single-page homepage only.
Global state	None	Local state plus one homepage data hook.
Automated testing	None configured	Build/type validation only.

Data flow

1. **App mount:** useHomeData starts a single homepage load with an AbortController.
2. **Parallel requests:** loadHomeData calls /homepage, /articles?per_page=40, /categories?per_page=50 and /tags?per_page=50.
3. **Normalisation:** API article summaries are mapped into display models with cleaned HTML entities, formatted dates, relative times, category fallbacks and local image fallbacks.
4. **Composition:** HomePage receives the assembled HomeData object and passes focused arrays to each section component.
5. **Search:** opening the search dialog performs a debounced call to /articles?search=...&per_page=6 and maps the returned summaries into compact result rows.
6. **Header:** the same loaded tag data powers the dialog's suggested "trending searches", so a separate header request is not needed.

The complete source-file map, including data, styling, assets and build files, is provided in Appendix A.

FRONTEND IMPLEMENTATION

Components, interactions and state

Core components

COMPONENT	RESPONSIBILITY	STATE / API BEHAVIOUR
Header	Utility bar, navigation, search entry, theme switch, mobile menu trigger.	Local search/menu state; live topic suggestions from homepage data.
SearchDialog	Modal search with debounce, loading skeleton, result rows, empty/error states and suggested tags.	280ms debounce; aborts stale requests; live /articles?search=.
BreakingTicker	Rotating breaking headlines and timestamps.	Auto-advances every 7 seconds; controls hidden for a single item.
Hero	Lead story presentation and carousel.	Carousel controls only render when more than one lead story exists.
TopStoriesRail	Four supporting stories alongside the lead.	Receives mapped article props.
LatestNews / TrendingList / Popular / ForYou	Editorial article shelves with different layouts.	Receive API-derived arrays and render images with fallbacks.
TrendingTopics	Tag chips with a featured accent.	Derived from the public tags endpoint.
Categories	Eight image tiles for featured sections.	Curated from live categories with local category artwork.
MediaStories	Legacy video/photo layout using mock assets.	Retained in the codebase but no longer rendered live.
Newsletter / Footer / ads	Brand, conversion and advertising surfaces.	Static UI; newsletter and ad destinations are placeholders.

Loading and failure behaviour

INITIAL LOAD

- Full skeleton keeps header, ad, hero and rail space stable.
- Fetch requests are aborted if the component unmounts.
- The page reveals module sections only when data exists.

ERROR PATH

- Dedicated connection-problem screen with API message.
- “Try again” increments a request version and refetches.
- Search errors stay inside the dialog and offer a retry by typing again.

Image fallback system

The API exposes absolute featured-image URLs. During review, every tested article image URL returned HTTP 404. A shared SmartImage component now catches load failures and swaps in a local fallback chosen from the article category. The homepage therefore remains visually complete while the backend storage issue is fixed. This is a resilience measure, not a substitute for the real article images.

API INTEGRATION

Live production contract and data mapping

Contract and base URL

Specification	OpenAPI 3.1; local contract at openapi.yaml; version 1.0.0.
Base URL	https://pebnews.pebnews.com/api/v1
Health check	GET /health returned HTTP 200 with {"status":"ok"}.
Response shape	Public collections use data, meta and links; homepage uses a single data object.
Limit	per_page is capped at 50 by the contract.
Errors	HTTP 422 validation errors and HTTP 429 rate limits are documented.

Endpoints used by the current frontend

FRONTEND NEED	REQUEST	USE
Homepage modules	GET /homepage	Breaking, lead and latest arrays.
Supporting stories	GET /articles?per_page=40&sort=newest	Top rail, trending, popular and For You shelves; also used to deduplicate lead/latest items.
Featured categories	GET /categories?per_page=50	Curated eight-tile category strip.
Trending topics	GET /tags?per_page=50	Eight topic chips and search suggestions.
Search dialog	GET /articles?search=...&per_page=6&sort=newest	Debounced live search results.

Normalisation rules

- Titles, categories and excerpts are stripped of HTML and decoded from entities such as & and '.
- Excerpts that begin with the article title followed by a dash are shortened to the remaining summary text.
- Publication dates are formatted as short dates for editorial bylines and as relative times for cards and tickers.
- Duplicate article slugs are removed before module allocation.
- Items already used in lead/latest/breaking are reserved so supporting shelves remain visually distinct where possible.
- Null image URLs and failed remote images fall back to local category artwork.

Current module mapping

HOMEPAGE MODULE	API SOURCE	SELECTION RULE
Breaking ticker	/homepage → breaking_news	First five unique items.
Hero	/homepage → lead_stories	All returned lead stories; only one currently seeded.
Latest News	/homepage → latest_news	Up to four, excluding lead slugs.
Top Stories rail	/articles	Next four unique articles after reserved items.
Trending	/articles	Next five titles from the newest article pool.
Popular	/articles	Next four articles; not a true popularity metric.
For You	/articles	Next four articles; not personalised until reader auth exists.
Trending Topics	/tags	First eight tags, first tag visually featured.
Featured Categories	/categories	Preferred order: The Nation, Business, Politics, Oil & Gas, Sports, Education, Entertainment, Health.

CORS and proxy strategy

Direct API responses advertise Access-Control-Allow-Origin: https://pebnews.com and allow credentials. Browser requests from localhost or arbitrary Netlify preview domains are therefore blocked. The project solves this in two layers:

- **Development:** Vite proxies /api to https://pebnews.pebnews.com.
- **Netlify:** netlify.toml rewrites /api/v1/* to the production API with status 200.
- **Configuration:** VITE_API_BASE_URL defaults to /api/v1; an absolute API URL can be supplied at build time if CORS is later opened.

Reader authentication status

The contract documents Laravel Sanctum cookie-session authentication rather than bearer tokens. Stateful flows require GET /sanctum/csrf-cookie on the backend origin, credentialed requests, the session cookie and an X-XSRF-TOKEN header. The current frontend only consumes public GET endpoints and does not yet implement registration, login, profile, bookmarks or CSRF bootstrapping. The deployed backend reports PHP 8.5.12 on Hostinger; CMS authentication is separate from reader authentication.

CONTENT SNAPSHOT

What the live API currently returns

METRIC	OBSERVED VALUE	COMMENT
Articles	210 total	Paginated public feed; 40 newest requested for homepage composition.
Categories	15	Includes Agriculture, Business, Crime, Editorial, Education, Electricity, Entertainment, Health, Metro, News, Oil & Gas, Politics, Religion, Sports and The Nation.
Tags	20	Includes #analysis, #art, #climate, #economy, #energy, #environment, #essays, #fashion and others.
Breaking news	5 items	All returned with the Breaking News label.
Lead stories	1 item	“Religion: Where we go from here”; the hero carousel correctly hides arrows for one item.
Latest news	4 items	Crime, The Nation, Oil & Gas and Agriculture.
Article images	HTTP 404	Every tested /storage/articles/... file was unavailable at review time.

Content quality observations

- The backend is populated with demo seed content. Many article titles share the pattern “Where we go from here” or “The everyday realities behind ... headlines”.
- Excerpts are duplicated from titles and truncated by the API; the frontend removes the repeated title prefix where possible.
- Reporter photos are null in the seed data, so the interface uses a local author avatar fallback.
- Published dates span late October to early November 2026; relative labels are generated at render time.
- Remote image captions exist in the API, but the image files themselves are missing from the public storage path.

> **Backend action required:** repair public storage for article images, or change the API’s featured-image URL generation to point to files that are actually reachable. Until then, the frontend will continue to show local category artwork rather than CMS media.

DESIGN SYSTEM

Visual language and responsive behaviour

Brand and colour

TOKEN	VALUE	USE
Header navy	#003078	Primary masthead and navigation surface.
Deep navy	#001030	Dark page and footer depth.
Brand red	#e60325	Breaking label, featured chips and accent rules.
Action blue	#0060f8	Links, active controls and calls to action.
Light page	#ffffff	Light-mode reading surface.

Typography

Headlines use Source Serif 4 through Georgia fallbacks; interface copy uses Inter through system sans-serif fallbacks. Typography was deliberately enlarged across the site during the final polish pass: the category eyebrow is 11px mobile / 12px small-screen, section titles are 24px / 26px, and article, metadata, navigation, footer, advertising and mobile menu text were increased for readability. Headline sizes remain editorial, with the lead story reaching 48px on wide screens.

Responsive layout

WIDTH	BEHAVIOR
390px	Single-column modules; hamburger navigation; ticker date compressed; newsletter card in flow; no horizontal overflow measured.
1024px	Desktop navigation, three-column news grid, hero rail visible and full advertising modules.
1440px	Wide editorial canvas with 1280px max content width and balanced hero/news hierarchy.

Theme behaviour

- Dark theme is the default.
- Theme choice persists in localStorage under peb-theme.
- ?theme=light and ?theme=dark override the saved preference for review links.
- Theme bootstrapping happens in a small inline script before React mounts, avoiding a flash of the wrong theme.
- The selected theme updates the browser color-scheme.

Advertising and editorial styling

The layout reserves distinct positions for a leaderboard, a mobile top unit, a full-width build/infrastructure unit, a riverside unit, a green-energy unit and a hotel unit. All are labelled “Advertisement” and use local artwork. They are presentation placeholders until an ad server or campaign destinations are supplied.

BUILD AND DEPLOYMENT

Commands, configuration and release path

Available scripts

```
pnpm dev       Start the Vite development server
pnpm build     Type-check and build to dist/
pnpm preview   Serve the production build locally
pnpm typecheck Run TypeScript without building
```

Environment configuration

```
VITE_API_BASE_URL=/api/v1
```

The value is relative by default so that both the Vite development proxy and the Netlify rewrite can forward requests. If an absolute API URL is used instead, the backend CORS configuration must permit the frontend origin.

Vite proxy

```
/api  ->  https://pebnews.pebnews.com
```

The proxy is configured for both the development server and preview server. This avoids browser CORS restrictions while preserving the production API contract and paths.

Netlify configuration

```
[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
  from = "/api/v1/*"
  to = "https://pebnews.pebnews.com/api/v1/:splat"
  status = 200
  force = true
```

Deployment options

1. **Netlify Drop:** upload the generated dist/ folder. The redirect file must be included in the deployment.
2. **Netlify Git build:** connect the project and use build command npm run build with publish directory dist.
3. **Custom production origin:** if deploying to https://pebnews.com, the backend already permits that origin; direct API calls can replace the proxy by setting an absolute VITE_API_BASE_URL.

> **Authentication caveat:** the current Netlify rewrite is suitable for public GET content. Reader login, CSRF cookies and bookmarks require a deliberate same-site cookie strategy and should not be routed through the proxy without testing Sanctum's stateful-domain and cookie settings.

VERIFICATION

What was tested and what remains uncertain

CHECK	RESULT	NOTES
TypeScript/Vite production build	Pass	pnpm build completes and emits the production bundle.
API health	Pass	GET /health returns {"status":"ok"}.
Homepage contract	Pass	5 breaking, 1 lead and 4 latest items were observed.
Article list contract	Pass	Pagination and total count were observed through the API.
Categories and tags	Pass	15 categories and 20 tags were returned.
Search endpoint	Pass	Live query returned matching articles through the proxy.
Development/Netlify proxy	Pass	API requests succeed through /api/v1.
Desktop rendering	Pass	Live data rendered at 1440px with fallback images.
Mobile layout measurement	Pass	At 390px viewport, document width remained inside the viewport; only a decorative newsletter bubble intentionally overflows inside a clipped container.
Dark/light visual review	Pass	Both themes were reviewed during the typography pass; theme bootstrapping is intact.
Remote article images	Fail	Tested image URLs returned HTTP 404; local fallback is active.
Search dialog with live keyboard input	Partial	API search logic and types are implemented; the dialog was not exhaustively exercised across devices.
Automated regression tests	Not run	No test suite exists.

Release readiness assessment

READY NOW

- Homepage demonstration on Netlify or the allowlisted domain.
- Live editorial content review.
- Responsive design presentation.
- API contract demonstration and search demo.

NOT READY YET

- Public launch with real media expectations.
- Article reading journeys.
- Reader accounts and saved stories.
- Ad or newsletter conversion flows.

RISKS AND LIMITATIONS

Known issues, severity and ownership

SEVERITY	ISSUE	IMPACT	OWNER
High	Article image URLs return 404.	No real CMS imagery is visible; local category art is shown instead.	Backend / storage
High	CORS only allows https://pebnews.com.	Direct browser API calls fail from other origins; proxies are required.	Backend + deployment
Medium	No article-detail route.	Headlines, search results and “View all” links go to #.	Frontend
Medium	No reader auth UI.	Sanctum capabilities are unused; the header displays a hardcoded user.	Frontend
Medium	No video/photo API.	The media section is hidden; potential editorial richness is lost.	Backend + product
Medium	Trending, Popular and For You lack true signals.	They are deterministic slices of the newest article pool, not analytics or personalisation.	Backend + frontend
Low	Demo content is repetitive.	Reviewers see repeated headline patterns and sparse excerpts.	Content / CMS
Low	Placeholder interactions.	Newsletter, ads, account chip, nav and footer links are static.	Frontend + product
Low	No automated tests or CI.	Regression protection depends on manual checks.	Engineering
Low	No 429 backoff.	Rate-limit errors surface as generic retry states.	Frontend

Technical debt

- Legacy mock data and the MediaStories component remain in the repository though they are no longer part of the live homepage.
- Header account and weather/location values remain hardcoded.
- All navigation and editorial links need a routing strategy.
- The frontend currently requests up to 40 articles to populate several supporting shelves; a dedicated editorial modules API would be more efficient and semantically correct.
- Image fallback requests still attempt the broken remote image first on every page load.

RECOMMENDED PLAN

Next steps, in priority order

Priority 0 — unblock production content

1. Repair Laravel public storage or the configured storage URL so article images return HTTP 200.
2. Re-test every seeded article image after the fix; keep the local fallback as a safety net.
3. Confirm the intended production frontend origin and either retain the proxy or update the API CORS allowlist.
4. Replace demo articles, excerpts and reporter data with publishable editorial content.

Priority 1 — complete the reading journey

1. Add React Router and an article-detail page using GET /articles/{slug}, including SEO metadata and canonical URLs.
2. Wire story cards, search results and category tiles to real routes.
3. Build category, tag and reporter archives using the existing public endpoints.
4. Add a search-results page for longer result sets beyond the dialog.

Priority 2 — reader accounts and product features

1. Implement Sanctum CSRF bootstrapping, login, registration, logout and /auth/me.
2. Replace the hardcoded account chip with a real signed-in or signed-out state.
3. Add bookmarks, profile and email/Google account flows where appropriate.
4. Resolve cookie-domain behaviour for authentication behind any Netlify proxy.

Priority 3 — editorial and operational maturity

1. Add media/gallery endpoints or CMS labels so video and photo modules can return.
2. Introduce real popularity/recommendation signals or relabel the derived shelves.
3. Connect the newsletter form to a backend or email provider.
4. Add automated tests for API mapping, search debounce, fallback images and responsive sections.
5. Initialize Git, define a CI pipeline and document a release checklist.
6. Add caching, request backoff for 429 responses and image optimisation/CDN delivery.

> **Recommended immediate release:** fix storage images, deploy the current homepage build through Netlify, and present it as a live homepage prototype. Treat article pages, authentication and media galleries as the next development milestone.

APPENDIX A

Project file map

PATH	PURPOSE
`openapi.yaml`	Authoritative OpenAPI 3.1 contract for the public and reader APIs.
`package.json`	Scripts and dependency manifest.
`vite.config.ts`	React/Tailwind plugins and development/preview API proxy.
`netlify.toml`	Netlify build command, publish directory and API rewrite.
`.env.example`	Relative API base URL configuration.
`index.html`	Document metadata, fonts, favicon and theme bootstrap.
`src/App.tsx`	Page shell, skip link, header, homepage and footer.
`src/main.tsx`	React root with StrictMode.
`src/pages/HomePage.tsx`	Homepage module composition, skeleton and error state.
`src/hooks/useHomeData.ts`	Homepage request lifecycle and retry.
`src/lib/api.ts`	Typed API client, base URL and error handling.
`src/lib/api-types.ts`	OpenAPI-aligned TypeScript models.
`src/lib/home-data.ts`	Homepage data aggregation and module allocation.
`src/lib/article.ts`	Display mapping, time formatting and text cleanup.
`src/lib/images.ts`	Category artwork and image fallbacks.
`src/types.ts`	UI view-model contracts including HomeData.
`src/data/mock.ts`	Navigation, static ads/footer data and legacy mock content.
`src/index.css`	Theme tokens, typography scale, container and shared styles.
`src/components/`	Header, search, sections, cards, ads, newsletter, footer and shared UI.
`dist/`	Current deployable production build.

Static and local assets

- public/images/ contains hero, story, category, ad, avatar, video and gallery placeholders.
- public/peb-news-logo.svg provides the favicon.
- No API secrets or private tokens are present in the frontend source.

APPENDIX B

API surface reference

GROUP	ENDPOINTS
System	GET /health, GET /openapi.yaml
Homepage	GET /homepage, GET /homepage/breaking-news, GET /homepage/lead-stories, GET /homepage/latest-news
Articles	GET /articles, GET /articles/{slug}
Taxonomy	GET /categories, GET /categories/{slug}, GET /tags, GET /tags/{slug}, GET /labels, GET /labels/{slug}, GET /labels/{slug}/articles
People	GET /reporters, GET /reporters/{slug}
Reader auth	POST /auth/register, GET /auth/email/verify/{id}/{hash}, POST /auth/email/verification-notification, POST /auth/login, POST /auth/logout, GET /auth/me, POST /auth/password/forgot, POST /auth/password/reset, GET /auth/google/redirect, GET /auth/google/callback
Reader profile	GET/PATCH/DELETE /profile, POST /profile/email, POST /profile/email/verify, POST /profile/google/connect, DELETE /profile/google/disconnect
Bookmarks	GET /bookmarks, POST /articles/{slug}/bookmark, DELETE /articles/{slug}/bookmark

Query and auth conventions

- Pagination: page and per_page; per-page values above 50 are clamped.
- Article filters: category, tag, reporter, search and sort=newest|oldest.
- Reader auth: Sanctum cookie session; CSRF cookie at /sanctum/csrf-cookie on the backend origin.
- Stateful requests require credentials and X-XSRF-TOKEN; CMS and reader sessions are separate.
- Unknown taxonomy slugs return an empty collection; unpublished or unavailable resources return 404.

APPENDIX C

Operational checklist

Before every release

1. Run pnpm build and confirm TypeScript and Vite complete successfully.
2. Open the homepage at 390px, 1024px and 1440px in both themes.
3. Confirm breaking, lead, latest, categories, tags and search return live data.
4. Check that no article card shows an empty image; confirm fallback behaviour if storage is still broken.
5. Test the search dialog with a known term and a no-result term.
6. Confirm the Netlify redirect proxy is present and that API requests are same-origin.
7. Verify no debug pages, test assets or temporary files are included in dist/.

Accessibility spot checks

- Confirm the skip link, semantic landmarks and visible focus styles.
- Open the search dialog with the keyboard; verify focus, Escape-to-close and scroll locking.
- Check carousel and ticker controls for accessible names and hidden controls.
- Confirm the mobile menu and dialog restore body scrolling when closed.
- Test with reduced motion enabled and review both theme variants.

Environment quick reference

Local development	pnpm dev · Vite proxies /api to production.
Production build	pnpm build · output in dist/.
Local preview	pnpm preview · preview proxy is configured.
Netlify	Build with npm run build; publish dist.
API base	https://pebnews.pebnews.com/api/v1
API proxy path	/api/v1

Bottom line

The project has moved from a static, mock-driven homepage to a live API-connected newsroom frontend. The core homepage experience, responsive design, themes, search and data handling are in place and build successfully. The most important remaining work is not visual polish; it is backend media delivery, article navigation, authentication and the transition from demo content to a complete editorial product.

Document generated from the current project state on 10 November 2026. API counts and content examples reflect the live production API at the time of review and may change as the CMS is updated.